Конспект лекции 1:
Введение в Deep Learning

Липкин С.М.

1 Что такое Deep Learning?

1.1 Определение и место в ML

Глубокое обучение (Deep Learning, DL) — подраздел машинного обучения, использующий многослойные нейронные сети для иерархического извлечения признаков из данных. В отличие от классического ML, где признаки конструируются вручную (feature engineering), DL учится представлять данные на разных уровнях абстракции автоматически.

Три ключевые идеи, на которых стоит DL:

1. **Сквозное обучение (end-to-end):** сырые данные подаются на вход, ответ получается на выходе — без ручного выделения признаков.

2. **Иерархия признаков:** первые слои учат простые паттерны (края, цвета), следующие — более сложные (текстуры, формы), последние — абстрактные концепции (объекты, сцены).

3. **Масштабирование:** в отличие от классических алгоритмов, качество глубоких сетей продолжает расти с увеличением объёма данных и числа параметров (при достаточной регуляризации).

1.2 Архитектура нейронной сети

Нейронная сеть — это композиция параметрических функций f_θ, организованных в слои. Каждый слой выполняет линейное преобразование с последующей нелинейной активацией:

$$h^{(l)} = \sigma(W^{(l)} h^{(l-1)} + b^{(l)})$$

где $h^{(l)}$ — активации l-го слоя, $W^{(l)}$, $b^{(l)}$ — веса и смещения, σ — нелинейная функция активации.

Полная сеть: $y = f_\theta(x) = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}(x)$.

1.3 Универсальная аппроксимация

Теорема о универсальной аппроксимации (Cybenko, 1989; Hornik, 1991): нейронная сеть с одним скрытым слоем и нелинейной активацией может аппроксимировать любую непрерывную функцию на компакте с любой точностью **при достаточном числе нейронов**. Однако на практике глубокие сети (с большим числом слоёв) работают эффективнее широких — они требуют меньше параметров для той же выразительной способности.

2 Анатомия обучения

Обучение глубокой сети — это минимизация эмпирического риска:

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\theta(x^{(i)}), y^{(i)})$$

2.1 Функции потерь

- **Cross-Entropy (классификация):**

$$\mathcal{L}_{\text{CE}} = -\sum_{c=1}^{C} y_c \log \hat{y}_c$$

  Совместно с softmax-слоем даёт хорошо калиброванные вероятности.

- **Mean Squared Error (регрессия):**

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- **Hinge Loss (SVM-like классификация):**

$$\mathcal{L}_{\text{hinge}} = \max(0, 1 - y \cdot \hat{y})$$

2.2 Градиентный спуск

Основной метод оптимизации — градиентный спуск. Существует три варианта:

1. **Batch GD:** полный проход по всем данным перед обновлением. Точно, но медленно, не подходит для больших датасетов.

2. **SGD (Stochastic GD):** обновление по одному случайному примеру. Максимально шумно, но быстро.

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})$$

3. **Mini-batch SGD:** компромисс — усредняем градиент по небольшому батчу (32–512 примеров). Стандарт на практике.

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{|B|} \sum_{i \in B} \nabla_\theta \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})$$

2.3 Обратное распространение (Backpropagation)

Градиент $\nabla_\theta \mathcal{L}$ вычисляется через цепное правило — последовательное дифференцирование от выхода сети к входу. Для L-слойной сети:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial h^{(L)}} \cdot \frac{\partial h^{(L)}}{\partial h^{(L-1)}} \cdots \frac{\partial h^{(l+1)}}{\partial h^{(l)}}}_{\text{backward pass}} \cdot \frac{\partial h^{(l)}}{\partial W^{(l)}}$$

Ключевая особенность: градиент — это **произведение** матриц Якоби промежуточных слоёв, что напрямую приводит к проблеме исчезающих/взрывающихся градиентов.

3 Проблемы глубоких сетей

3.1 Проблема 1: Исчезающие и взрывающиеся градиенты

Математическая причина: градиент на l-м слое содержит произведение $L - l$ матриц:

$$\frac{\partial \mathcal{L}}{\partial h^{(l)}} = \frac{\partial \mathcal{L}}{\partial h^{(L)}} \prod_{k=l+1}^{L} \mathrm{diag}(\sigma'(z^{(k)}))(W^{(k)})^T$$

Если $\|\mathrm{diag}(\sigma'(z))W^T\| < 1$ — произведение экспоненциально затухает (исчезающий градиент, ранние слои не обучаются). Если > 1 — экспоненциально растёт (взрывающийся градиент, NaN, нестабильность).

Классический пример: sigmoid-активация имеет максимальную производную $\sigma'(x) \leq 0.25$. В 10-слойной сети градиент затухает как $0.25^{10} \approx 10^{-6}$.

Последствия:

- Ранние слои почти не меняются $\rightarrow$ сеть не учится иерархии признаков.
- Глубокие сети (10+ слоёв) невозможно обучить стандартным SGD и sigmoid.

3.2 Проблема 2: Переобучение (Overfitting)

Когда число параметров сети значительно больше числа примеров в датасете, сеть может «выучить» шум в данных вместо истинных закономерностей.

Симптомы:

- Loss на обучающей выборке стремится к нулю.
- Loss на валидационной выборке перестаёт падать и начинает расти.
- Большой разрыв между train и validation метриками.

Факторы риска:

- Мало данных относительно числа параметров.
- Шум в разметке (label noise).
- Избыточная ёмкость модели (capacity).

3.3 Проблема 3: Седловые точки и локальные минимумы

В многомерных пространствах (10^6–10^{12} параметров) седловые точки встречаются **гораздо чаще**, чем локальные минимумы. Доля локальных минимумов среди стационарных точек экспоненциально мала (Dauphin et al., 2014).

В седловой точке $\nabla f(\theta) = 0$, но матрица Гессе имеет и положительные, и отрицательные собственные значения. SGD «застревает» в седловой точке, потому что градиент в ней нулевой.

Современное понимание (Choromanska et al., 2015): в достаточно больших сетях **большинство локальных минимумов имеют значение функции потерь, близкое к глобальному минимуму**. Проблема не в локальных минимумах, а в седловых точках.

3.4 Проблема 4: Внутренний ковариационный сдвиг (Internal Covariate Shift)

Определение: распределение входов каждого слоя постоянно меняется в процессе обучения, потому что меняются веса предыдущих слоёв.

Последствия:

- Приходится использовать низкий learning rate, чтобы компенсировать нестабильность.
- Необходима осторожная инициализация весов.
- Зависимость от порядка и масштаба данных.

3.5 Проблема 5: Вычислительная сложность и подбор гиперпараметров

- Обучение одной модели может занимать дни и недели на кластере GPU.
- Число гиперпараметров велико: learning rate, архитектура, размер батча, моментум, weight decay, dropout rate, расписание lr и т.д.
- Поиск по сетке (grid search) экспоненциален. На практике: random search, Bayesian optimization, или наследование известных рецептов.

4 Методы решения

4.1 Решение 1: Продвинутые оптимизаторы

4.1.1 SGD с Momentum

Идея: накапливаем инерцию градиента, как шар, катящийся с горы.

$$\begin{aligned} v_t &= \gamma v_{t-1} + \eta \nabla \mathcal{L}(\theta_t) \\ \theta_{t+1} &= \theta_t - v_t \end{aligned}$$

где $\gamma \in [0, 1)$ — коэффициент затухания (обычно 0.9).

Эффект: Momentum сглаживает траекторию, подавляет осцилляции в узких оврагах loss landscape и ускоряет сходимость. Позволяет «проскакивать» мелкие локальные минимумы и седловые точки.

4.1.2 Adam (Adaptive Moment Estimation)

Kingma & Ba, 2015. Комбинация Momentum (первый момент) с адаптивным learning rate (второй момент):

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \end{aligned}$$

Параметры: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Преимущества:

- Каждый параметр имеет свой адаптивный learning rate.
- Работает «из коробки» на широком классе задач.
- Устойчив к шумным градиентам.

4.1.3 Сравнение оптимизаторов

- **SGD:** простота, хорошее обобщение, но медленная сходимость.
- **SGD + Momentum:** быстрее SGD, хорошо обобщает.
- **Adam:** быстро сходится, работает «из коробки», но иногда хуже обобщает.
- **AdamW (Loshchilov & Hutter, 2019):** Adam с корректным weight decay (не путать с L2-регуляризацией в Adam).
- **RMSprop:** предшественник Adam, только адаптивный lr.

4.2 Решение 2: Правильные функции активации

4.2.1 Sigmoid и Tanh (классические)

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Недостаток: насыщение при $|x| > 2$ (sigmoid) / $|x| > 3$ (tanh) $\to$ производная $\to 0 \to$ исчезающий градиент.

4.2.2 ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

Преимущества:

- Производная 1 для $x > 0$ — нет насыщения, градиент проходит.
- Вычислительно дёшево: один сравнение.
- Спасичная (sparse) активация — многие нейроны дают 0.

Недостаток: «умирающий ReLU» (dying ReLU) — если нейрон перешёл в область $x < 0$, его градиент равен 0, и он больше никогда не восстановится.

4.2.3 Улучшенные варианты ReLU

- **Leaky ReLU:** $f(x) = \max(\alpha x, x)$ с $\alpha \approx 0.01$. Небольшой градиент для $x < 0$.
- **PReLU (Parametric ReLU):** α — обучаемый параметр.
- **ELU:** $f(x) = \max(x, \alpha(e^x - 1))$ — плавное затухание для $x < 0$.
- **Swish/SiLU (Ramachandran et al., 2017):** $f(x) = x \cdot \sigma(x)$ — найдена автоматическим поиском. Гладкая, немонотонная.
- **GELU (Hendrycks & Gimpel, 2016):** $f(x) = x \cdot \Phi(x)$, где Φ — CDF нормального распределения. Используется в BERT, GPT, ViT.

4.3 Решение 3: Инициализация весов

Неправильная инициализация весов — одна из главных причин проблем с градиентами. Правильная инициализация должна сохранять дисперсию сигнала (и градиента) между слоями.

4.3.1 Xavier/Glorot Init (2010)

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

Условие: $\text{Var}(y) = \text{Var}(x)$ для линейной части. Вывод:

$$\text{Var}(y) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i x_i) = n_{\text{in}} \,\text{Var}(w)\,\text{Var}(x) \implies \text{Var}(w) = \frac{1}{n_{\text{in}}}$$

Аналогично для обратного прохода: $\text{Var}(w) = 1/n_{\text{out}}$. Компромисс: $\text{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$.

Подходит для sigmoid и tanh, но **не подходит** для ReLU.

4.3.2 He/Kaiming Init (2015)

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

Учитывает, что ReLU «убивает» половину нейронов, поэтому дисперсия должна быть в 2 раза больше, чем у Xavier. Оптимальна для ReLU и его вариантов.

4.3.3 Практические рекомендации

- ReLU + He init: стандартная комбинация для свёрточных и полносвязных сетей.
- Tanh + Xavier: для RNN (но лучше использовать LayerNorm).
- Эпоха больших языковых моделей: для Transformer обычно используется $\mathcal{N}(0, d_{\text{model}}^{-1/2})$.

4.4 Решение 4: Batch Normalization

Ioffe & Szegedy, 2015. Одна из самых влиятельных статей в DL.

4.4.1 Алгоритм

На каждом шаге нормализуем активации по мини-батчу:

$$\mu_B = \frac{1}{|B|}\sum_{i=1}^{|B|} x_i, \quad \sigma_B^2 = \frac{1}{|B|}\sum_{i=1}^{|B|}(x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

где γ (масштаб) и β (сдвиг) — обучаемые параметры, восстанавливающие выразительную способность слоя.

На инференсе: μ и σ заменяются на скользящие средние, накопленные во время обучения.

4.4.2 Эффекты

- Позволяет использовать значительно **более высокий learning rate** (до 10x).
- Снижает зависимость от инициализации.
- Действует как **регуляризатор** (добавляет шум от оценки статистик по мини-батчу).
- Ускоряет сходимость в 10+ раз.

4.4.3 Механизм

Современное понимание (Santurkar et al., 2018): BN улучшает не столько распределение активаций, сколько делает loss landscape **более гладким** — градиенты становятся более предсказуемыми и менее чувствительными к масштабу весов.

4.4.4 Альтернативы

- **Layer Normalization (LayerNorm):** нормализация по признакам (не по батчу). Стандарт для Transformer, RNN. Не зависит от batch size.
- **Instance Normalization:** для задач переноса стиля.
- **Group Normalization:** для задач с маленькими батчами (сегментация, детекция).
- **RMS Normalization:** упрощение LayerNorm (деление на $\sqrt{\text{RMS}}$). Используется в Llama.

4.5 Решение 5: Регуляризация

4.5.1 L1/L2-регуляризация (Weight Decay)

$$\tilde{\mathcal{L}} = \mathcal{L} + \frac{\lambda}{2}\|\theta\|_2^2$$

Штраф за большие веса. L2 соответствует Gaussian prior на веса (MAP-оценка). L1 даёт разреженные веса (Laplace prior).

Важно: в Adam L2-регуляризация работает иначе (делится на адаптивный lr). AdamW — корректная реализация weight decay для Adam.

4.5.2 Dropout

Srivastava et al., 2014. На каждом проходе случайно «выключаем» долю p нейронов:

$$h_{\text{dropped}} = h \odot m, \quad m_i \sim \text{Bernoulli}(1-p)$$

Интуиция:

- Ensemble-эффект — каждая подсеть со случайным набором нейронов учится независимо.
- Модель не может полагаться на отдельные нейроны — все нейроны учатся давать полезные признаки.
- При инференсе масштабируем веса на $1-p$ (или используем inverted dropout).

На практике: $p \approx 0.2$ для рекуррентных слоёв, $p \approx 0.5$ для полносвязных. В свёрточных слоях используется Spatial Dropout (выключает целые каналы).

4.5.3 Data Augmentation

Искусственное расширение обучающей выборки путём применения случайных преобразований:

- **Изображения:** повороты, сдвиги, отражения, изменение цвета/яркости/контраста, обрезка (RandomCrop), смешивание (Mixup, CutMix).
- **Текст:** back-translation, word dropout, синонимы.
- **Аудио:** SpecAugment (маскирование по частоте и времени).

Почему работает: модель учится быть инвариантной к несущественным вариациям данных.

4.5.4 Early Stopping

Следим за метрикой на валидационной выборке. Когда val loss перестаёт уменьшаться (patience эпох), останавливаем обучение и возвращаем модель с лучшим val loss.

Преимущества: не требует модификации модели, бесплатно.

4.5.5 Label Smoothing

$$y_{\text{smooth}} = (1 - \varepsilon) y_{\text{one-hot}} + \frac{\varepsilon}{K}$$

где K — число классов, $\varepsilon \approx 0.1$.

Предотвращает переуверенность модели (overconfidence). Используется в Inception, Transformer, большинстве современных архитектур.

4.6 Решение 6: Планирование learning rate

Learning rate — самый важный гиперпараметр. По ходу обучения lr необходимо уменьшать.

4.6.1 Основные стратегии

- **Step decay:** уменьшаем lr в 10 раз каждые N эпох (например, каждые 30 эпох).
- **Exponential decay:** $\eta_t = \eta_0 \cdot \gamma^t$, $\gamma \approx 0.95$.
- **Cosine annealing (Loshchilov & Hutter, 2017):**

  $$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right)$$

  Плавное уменьшение по косинусу.
- **Reduce on Plateau:** уменьшаем lr в 2–10 раз, когда val loss перестаёт улучшаться.
- **One-cycle (Smith, 2018):** lr сначала растёт до максимума, потом плавно уменьшается. Позволяет обучить модель за меньшее число эпох.

4.6.2 Warmup

Если начать обучение с высоким lr, градиенты могут «улететь» (особенно для Transformer и Adam). Решение: на первых K шагах линейно растём lr от 0 до базового значения:

$$\eta_t = \eta_{\text{base}} \cdot \frac{t}{K}, \quad t = 1, \ldots, K$$

В оригинальном Transformer: $K = 4000$ шагов warmup.

4.6.3 Практический рецепт

Для современных архитектур: Cosine warmup + AdamW. Например: 10% эпох warmup, 90% cosine decay.

5 Сводка: таблица проблем и решений

Проблема	**Основные решения**
Исчезающий градиент	ReLU, He init, BatchNorm, Residual connections
Взрывающийся градиент	Gradient clipping, BatchNorm
Переобучение	Dropout, Weight Decay, Data Augmentation, Early Stopping
Медленная сходимость	Adam, Momentum, LR schedule
Внутренний ковариационный сдвиг	Batch/Layer Normalization
Плохая инициализация	He/Xavier init
Седловые точки	Momentum, Adam (инерция проскакивает)
Переуверенность модели	Label Smoothing

6 Заключение

6.1 Главный вывод

Глубокие сети работают не благодаря одному «волшебному» приёму, а благодаря **комбинации** хорошо подобранных техник:

Успех ≈ правильная инициализация+нормализация+продвинутый оптимизатор+регуляризация+

6.2 Исторический контекст

- 1980-е: Backpropagation (Rumelhart, Hinton, Williams).
- 2006: Deep Belief Networks (Hinton) — начало эпохи DL.
- 2012: AlexNet (Krizhevsky, Sutskever, Hinton) — победа на ImageNet.
- 2014: Dropout, Adam.
- 2015: Batch Normalization, ResNet.
- 2017: Transformer (Vaswani et al.).
- 2018–настоящее: BERT, GPT, Scaling Laws, Foundation Models.

6.3 Дальнейшие лекции курса

- Лекция 3: Свёрточные нейронные сети и Computer Vision.
- Лекция 4: Рекуррентные нейронные сети.
- Лекция 6: Transformer и механизмы внимания.